# Parallax — A Parallel VHDL Simulator

**Submission 1 – Progress Update** | 20 Mar 2026

**Name**: Chirag Kathpalia **Entry Number**: 2025MCS2098
**Course**: COD7001 – Cornerstone Project **Supervisor**: Prof. Kolin Paul

**Repo**: github.com/KathpaliaChirag/Parallax—A-Parallel-VHDL-Simulator
**Video Demo link (also available on github)**:
Gdrive.com/KathpaliaChirag/Demo-1-Parallax

## Project Timeline & Status

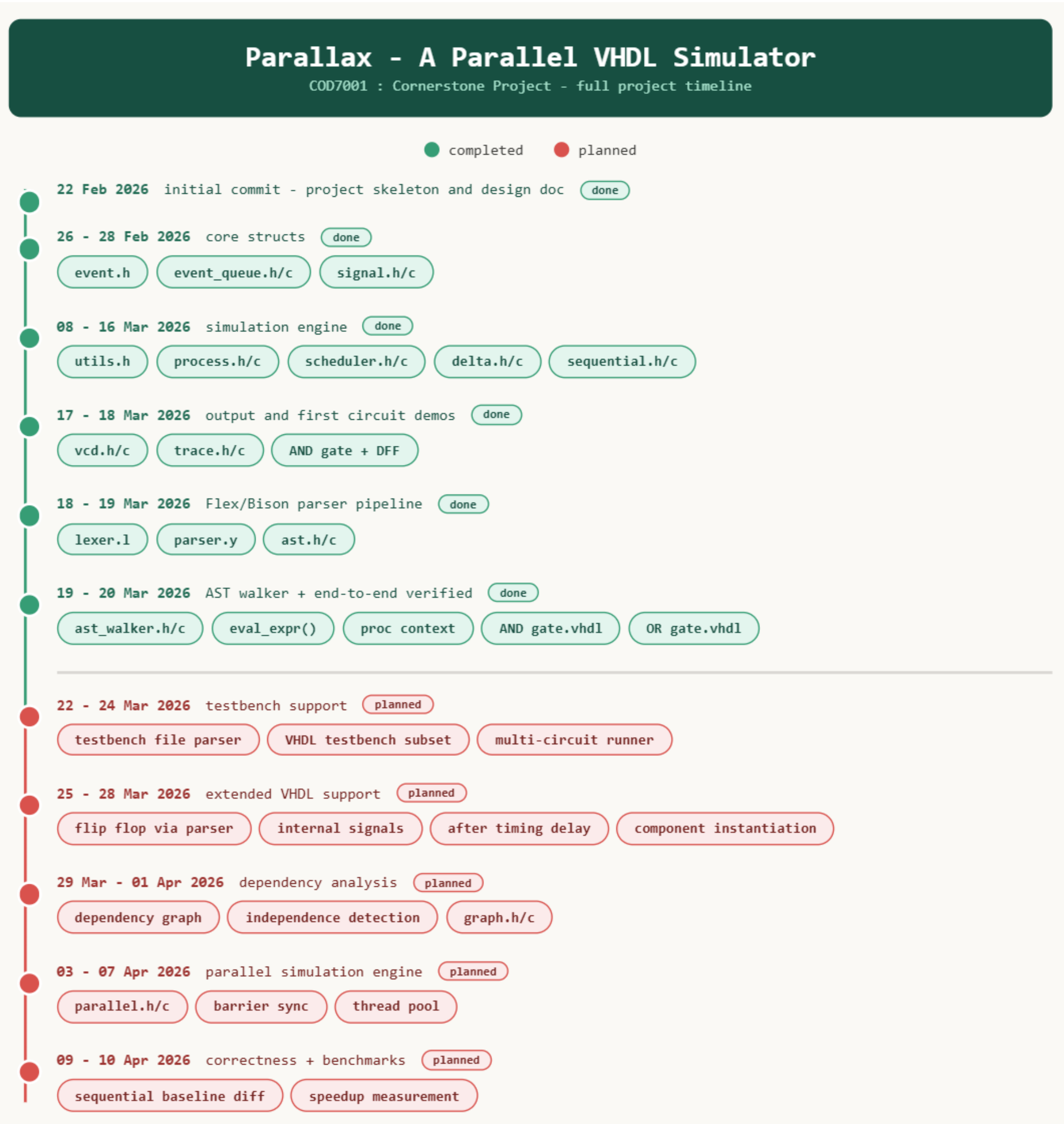


Figure 1: full project timeline - green = done, red = planned for submission 2

## System architecture

The diagram below shows how every module connects. A VHDL file enters at the left, passes through five layers, and exits as a verified VCD waveform.

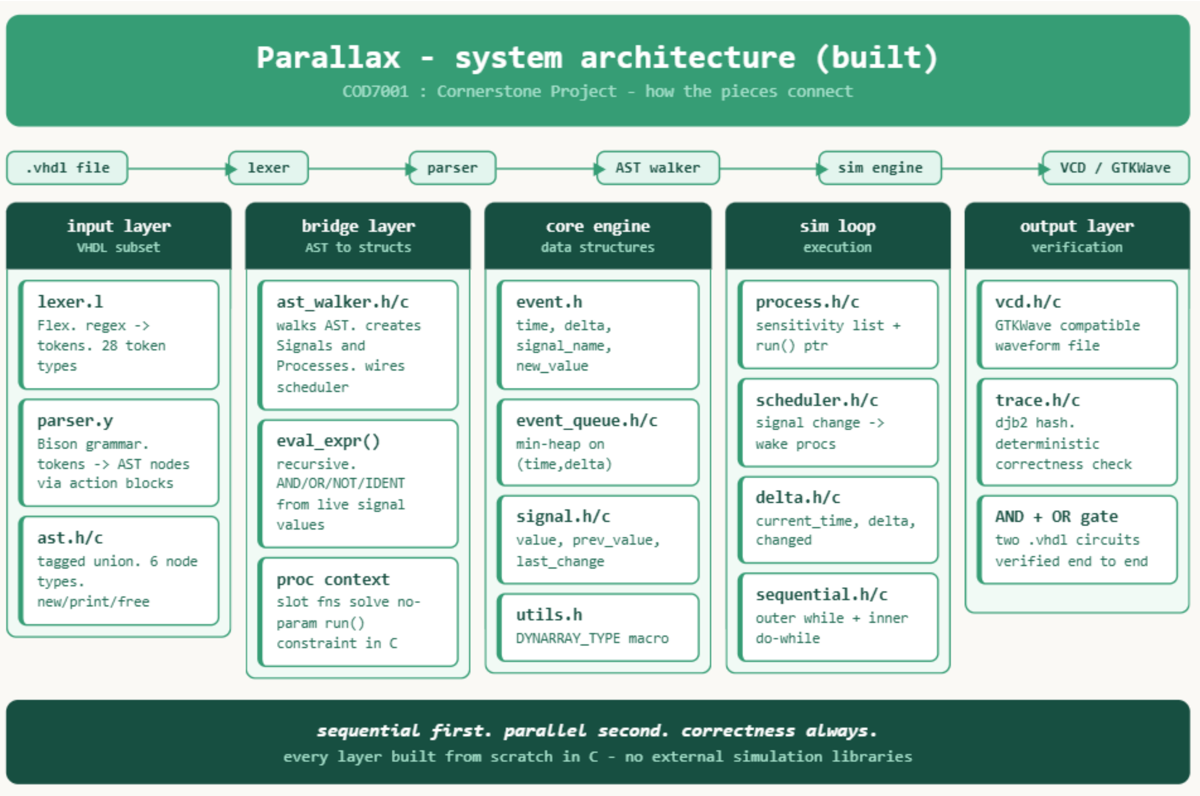


Figure 2: Parallax system architecture – five layers, all built from scratch

The five layers and what they do:

- **input layer** – `lexer.l` (Flex, 28 tokens) + `parser.y` (Bison grammar) + `ast.h/c` (tagged union tree with 6 node types). raw VHDL text in, `ast_entity` + `ast_root` out.
- **bridge layer** – `ast_walker.h/c` walks the AST and creates real `Signal` and `Process` structs. `eval_expr()` evaluates expression nodes recursively at runtime. a proc context system solves the no-param `run()` constraint in C using global slots.
- **core data layer** – `utils.h` (macro dynamic arrays), `event.h` (sim atom), `event_queue.h/c` (min-heap on `(time,delta)`), `signal.h/c` (value + prev_value). shared between sequential and parallel – this is by design.
- **execution layer** – `process.h/c` (sensitivity list + `run()` ptr), `scheduler.h/c` (signal change wakes matching procs), `delta.h/c` (global sim clock, zero-delay cycles), `sequential.h/c` (outer while on queue, inner do-while on delta).
- **output layer** – `vcd.h/c` (IEEE VCD format, GTKWave compatible), `trace.h/c` (djb2 hash over all transitions, hash `=` `1207120264`, sequential and parallel must match).

## what’s built

The diagram below shows every module built so far, grouped by when it was built, with commit hashes.

**.vhdl** input → **lexer.l** tokens → **parser.y** grammar → **ast.h/c** tree → **ast_walker** structs → **sequential** sim loop → **vcd + trace** output

26 FEB – 28 FEB 2026 · CORE STRUCTS

**src/core/ — foundations** data layer

**event.h**
the atom of simulation. every signal change is an Event with time, delta, signal_name, new_value
#617afa7 · 26 Feb

**event_queue.h/c**
min-heap on (time,delta). always pops earliest event. init_queue, insert_ele, extract_min
#a848170 · 26 Feb

**signal.h/c**
each wire in the circuit. value, prev_value, last_change_on. init, update, get_value
#c6fabe9 · 28 Feb

**src/core/ — engine** 08 – 16 Mar 2026

**utils.h**
DYNARRAY_TYPE(T) macro. ## token paste generates DynArray_Signal, DynArray_Event at compile time
#c776b03 · 08 Mar

**process.h/c**
Process with id, name, void(*run)(void) ptr and DynArray_Signal sensitivity list
#c9233fa · 10 Mar

**scheduler.h/c**
signal change → process wakeup. scheduler_notify calls run() on all matching procs
#8899248 · 11 Mar

**delta.h/c**
global sim clock. current_time, delta, changed. handles zero-delay cycles correctly
#bdc4d74 · 12 Mar

**sequential.h/c**
main sim loop. outer while on queue, inner do-while on delta. runs run_simulation()
#eef8021 · 16 Mar

17 MAR – 18 MAR 2026 · OUTPUT AND FIRST CIRCUITS

**src/output/** verification

**vcd.h/c**
IEEE VCD format. header declares signals with symbols. vcd_write_change logs each transition with #time. GTKWave verified
#cfeaebf · 17 Mar

**trace.h/c**
djb2 hash over all signal transitions. deterministic fingerprint. hash = 1207120264
#928c04e · 18 Mar

**hardcoded demos** proof of concept

**AND gate**
Y = A and B. global signal ptrs. process wakes on A or B. inserts Y event into queue
#5503039 · 17 Mar

**D flip flop**
rising CLK edge triggers Q = D. fixed critical heap bug in extract_min during this work
#a6502d6 · 17 Mar

→ these were **hardcoded C demos** to verify the sim engine. superseded on **20 Mar 2026** when AND gate and OR gate were both simulated directly from **.vhdl files** via the full parser pipeline


Figure 3: all built modules – grouped by phase with commit hashes

A quick honest note on where the interesting problems were:

**delta cycle infinite loop** (16 Mar) – `advance_time()` was running every iteration even when events remained at `current_time`, causing an infinite loop. fix was checking `p->data[0].time > current_time` before advancing. took a while to spot.

**AST walker zero signals** (19–20 Mar) – `ast_root` was set to the arch node only, entity node was lost. fix was storing `ast_entity = $1` in the grammar rule and calling `ast_walk(ast_entity, ...)` before `ast_walk(ast_root, ...)`. classic parser bug.

**proc context system** – C has no closures. `run()` takes no params. solved by storing target signal name + expression pointer in a global slot array. `run_proc_0/1/2/3` each call `run_proc_generic(idx)`. works up to 8 processes.

## commit history

```
PS C:\Users\chira\OneDrive\Desktop\Sem-1-2 Coursework\COP7001\GOD-PROJECT\Parallax---A-Parallel-VHDL-Simulator> git log --pretty=format:
"%h | %ad | %s" --date=format:"%d-%m-%Y %H:%M"
d53810f | 20-03-2026 22:32 | added OR gateb finally works
36d5c7c | 20-03-2026 22:00 | added working and finally yeaaaaaahhhhhh
0758f8b | 19-03-2026 01:26 | compilation works but signal fails at some places
14edd6a | 19-03-2026 01:06 | something broke lmao
fafd1bc | 18-03-2026 23:37 |  Flex/Bison parser workinnnnnn
928c04e | 18-03-2026 20:51 | Just finished with trace hashing
cfeaebf | 17-03-2026 15:05 | VCD output verified for AND gate and DFF in GTKWave
a6502d6 | 17-03-2026 04:36 | D flip flop working + fixed critical heap bug in extract_min
5503039 | 17-03-2026 00:15 | AND gate hardcoded and verified, simulation engine working end-to-end
f6ab005 | 16-03-2026 23:32 | updated task list
eef8021 | 16-03-2026 22:56 | feat: sequential simulation engine working with (time,delta) ordering
bdc4d74 | 12-03-2026 03:23 | Add delta module and integrate into main
7e9ec11 | 12-03-2026 00:34 | Support Event.delta ordering and docs
54791fa | 11-03-2026 22:10 | Add basic scheduler test and my_run
8899248 | 11-03-2026 21:48 | Add simple scheduler module and editor setting
a867fe2 | 10-03-2026 13:16 | added test of process
c9233fa | 10-03-2026 13:04 | added implementation of process
e24ed5a | 09-03-2026 19:31 | connected marcos implementation to eventqueue
c776b03 | 08-03-2026 15:30 | added dynmamic array implementation is macos
d186a69 | 08-03-2026 00:30 | updated my plan
59fde97 | 28-02-2026 17:08 | added a basic test to see signals work
c6fabe9 | 28-02-2026 16:53 | added basic signal init, uodate and read functions
47d89e3 | 28-02-2026 15:47 | Rename design_doc.pdf to 2025MCS2098_design_doc.pdf
edf1609 | 27-02-2026 02:39 | Rename and polish design doc, add PDF
848dc36 | 27-02-2026 02:19 | Add project skeleton, design doc, and README plan
a848170 | 26-02-2026 16:40 | Wprking event queue tested with manual push of elements
9299501 | 26-02-2026 16:27 | just made a function to extract min...seems its working too
1e26dc5 | 26-02-2026 15:17 | Added event struct, queue struct as well as initialisation
617afa7 | 26-02-2026 14:25 | just added an event struct with headerguards
861bac3 | 22-02-2026 01:14 | Initial commit
PS C:\Users\chira\OneDrive\Desktop\Sem-1-2 Coursework\COP7001\GOD-PROJECT\Parallax---A-Parallel-VHDL-Simulator> -LCompiler/build -lcompi
ler
```

Figure 4: full commit history – 28 commits, 26 Feb to 20 Mar 2026

## what's next – submission 2

**Parallax - planned for submission 2**

COD7001 : Cornerstone Project - target 10 Apr 2026 - submission 17 Apr 2026

| phase | target date | deliverables |
|---|---|---|
| testbench support | 22 - 24 Mar | testbench file parser · VHDL testbench subset · multi-circuit runner |
| extended VHDL | 25 - 28 Mar | flip flop via parser · internal signals · after timing delay · component instantiation |
| dependency analysis | 29 Mar - 01 Apr | dependency graph · independence detection · graph.h/c |
| parallel engine | 03 - 07 Apr | parallel.h/c · barrier sync · thread pool |
| correctness + benchmarks | 09 - 10 Apr | sequential baseline diff · speedup measurement |

*sequential first. parallel second. correctness always.*